



Python modules for advanced programming

Lecture 2: Functions and Loops

Lecturer: Dr. Havana Rika

Thanks: Dr. Orly Barzilai

Today

- ❖ Functions
- ❖ Global variables and constants.
- ❖ While Loop
- ❖ For Loop & range function

Functions

- In Python a function is defined using the **def** keyword.

- Syntax:

```
def function_name(parm1,parm2,...):  
    function content
```

- Example:

```
def myFunc():  
    print("Hello from a function")
```

```
def multiply(x,y):  
    z=x*y  
    return z
```

Calling a Function

```
def multiply(x,y):  
    z=x*y  
    return z
```

```
print(multiply(2,3))
```

```
def myFunc():  
    print("Hello from a function")  
  
myFunc()  
myFunc()
```

```
Hello from a function  
Hello from a function
```

```
def myFunc(name):  
    print("Hello " +name)  
  
myFunc("Emil")  
myFunc("Sivan")  
myFunc("Sani")
```

```
Hello Emil  
Hello Sivan  
Hello Sani
```

Override Function

```
def func(x,y):  
    return x + y
```

```
print(func(5, 7))
```

```
def func(x):  
    return x * x  
print(func(4))
```

```
print(func(1,2))
```

```
func = "Hello"  
print(func)  
print(func(2))
```

The main Function

```
1 # This program has two functions.
2 # define the main function.
3 def main():
4     print('I have a message for you.')
5     message()
6     print('Goodbye!')
7
8 # define the message function.
9 def message():
10    print('I am Grace Kelly,')
11    print('Princess of Monaco.')
12
13 # Call the main function.
14 main()
```

Program Output

```
I have a message for you.
I am Grace Kelly,
Princess of Monaco.
Goodbye!
```

Default Parameter Value

- Set a **default parameter** value when creating a function.
- If we call the function without argument, it uses the default value.

```
def power(x,y=2):  
    return x ** y
```

```
print(power(2,3))  
print(power(5))
```

```
def my_function(country = "Norway"):  
    print("I am from " + country)
```

```
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("Brazil")
```

```
I am from Sweden  
I am from India  
I am from Norway  
I am from Brazil
```

Keyword Arguments

- You can also send arguments with the **key = value** syntax.
- This way the order of the arguments does not matter.

```
def power(x,y):  
    return x ** y
```

```
print(power(3,6))
```

```
print(power(y=6,x=3))
```


Returning None

- The special value `None` means “no value”.
- Sometimes it is useful to return `None` from a function to indicate that an error has occurred.

```
def divide(num1, num2):  
    if num2 == 0:  
        result = None  
    else:  
        result = num1 / num2  
    return result
```

Returning Multiple Values

```
def get_name():  
    # Get the user's first and last names.  
    first = input('Enter your first name: ')  
    last = input('Enter your last name: ')  
    # Return both names.  
    return first, last
```

- When you call this function in an assignment statement, you need to use two variables on the left side of the = operator.

```
first_name, last_name = get_name()
```

Mutable Vs. Immutable

- **immutable** can not be changed its value once it is created.
- whenever we change the value, a new object will be created.

Type	Mutable
------	---------

String	X
--------	---

int	X
-----	---

Float	X
-------	---

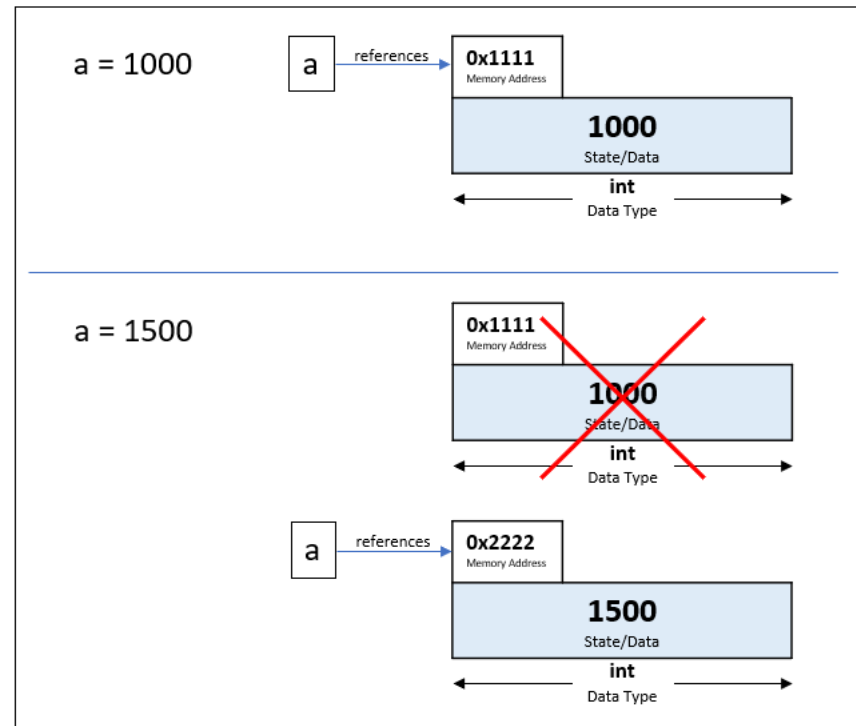
Complex	X
---------	---

List	V
------	---

Tuple	X
-------	---

Set	V
-----	---

Dictionary	V
------------	---



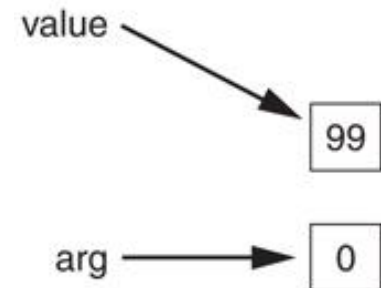
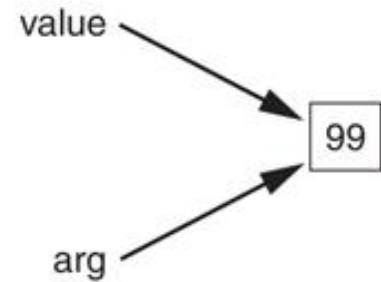
By Ref Or By Value?

- **All types** of data are **passed by Ref.**
 - But **immutable** acts like **by value**,
 - And **mutable** acts like **by ref.**

```
def main():  
    value = 99  
    print(f'The value is {value}.')  
    change_me(value)  
    print(f'Back in main the value is {value}.')
```

```
def change_me(arg):  
    print('I am changing the value.')  
    arg = 0  
    print(f'Now the value is {arg}.')
```

```
main()
```



Global Variables

- Variables that are created **outside of a function** are known as **global** variables.
- Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"

def myfunc():
    print("Python is " + x)

myfunc()
```

Python is awesome

Global Variables

- When you create a **variable inside** a function, that variable is **local**, and can only be used inside that function.
- To create a global variable inside a function, you can use the **global** keyword.

```
x = "awesome"

def myfunc():
    global x
    x = "fantastic"

myfunc()

print("Python is " + x)
```

Python is fantastic

```
x = "awesome"

def myfunc():
    x = "fantastic"
    print("Python is " + x)

myfunc()

print("Python is " + x)
```

Python is fantastic
Python is awesome

Constant Values

```
1 # The following is used as a global constant
2 VAT= 0.17
4
5 def main():
6     price= float(input('Enter the price: '))
7     discount = float(input('Enter the percentage of discount: '))
8     show_total_pay(price, discount)
10
14 def show_total_pay(price, discount):
15     totalAmount = price_after_discount(price, discount) * (1+VAT)
16     print('Total amount: $', format(totalAmount, ',.2f'), sep="")
19
23 def price_after_discount(price, discount):
24     return price*(1-discount/100)
28
29 # Call the main function.
30 main()
```

Program Output

Enter the price: **172.00** ↵

Enter the percentage of discount: **20** ↵

Total amount: \$160.99

While Loop

❑ execute a set of statements as long as the condition is true.

❑ Syntax:

while (test condition):
 do something
 [increment value]

```
i = 1
while i < 6:
    print(i)
    i += 1
```

1
2
3
4
5

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

```
i = 1
while i < 6:
    print(i)
    if (i == 3):
        break
    i += 1
```

1
2
3

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

1
2
4
5
6

```
1
2
3
4
5
i is no longer less than 6
```


For Loop

- A **for** loop is used for iterating over a sequence.
- Syntax:

for *variable* **in** [*var1,var2,var3...*]:
statements

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

```
apple  
banana  
cherry
```

```
for x in "banana":  
    print(x)
```

```
b  
a  
n  
a  
n  
a
```

```
for num in [1,2,3,4,5]:  
    print(num)
```

```
1  
2  
3  
4  
5
```

For Loop

- ❑ A **for** loop is used for iterating over a sequence.
- ❑ Syntax:

for *variable* **in** [*var1,var2,var3...*]:
statements

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        break
    print(x)
```

apple

```
for x in [1,2,3,4,5]:
    print(x)
else:
    print("Finally finished!")
```

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

apple
cherry

```
1
2
3
4
5
Finally finished!
```

The range Function

- `range(start=0, end, step=1)`
- returns a sequence of numbers,
 - starting from **start**,
 - increments by **step**,
 - and ends at **end** (not included).

```
for x in range(6):  
    print(x)
```

0
1
2
3
4
5

```
for x in range(2, 6):  
    print(x)
```

2
3
4
5

```
for x in range(2, 30, 3):  
    print(x)
```

2
5
8
11
14
17
20
23
26
29

Nested Loops

*#This program displays a rectangular pattern of **

```
rows=int(input("How many rows?\n"))
cols=int(input("How many columns?\n"))
```

```
for r in range(rows):
    for c in range(cols):
        print("*",end="")
    print()
```

Another Option

```
for r in range(rows):
    print(cols*"*)
```